



COLÉGIO
POLITÉCNICO
UFSC

UNIVERSIDADE FEDERAL DE SANTA MARIA

COLÉGIO POLITÉCNICO DA UFSC

CURSO SUPERIOR DE TECNOLOGIA EM SISTEMAS PARA INTERNET

MATHEUS GABRIEL FLECK DE MELLO

SISBOLSA — GESTÃO DE BOLSISTAS E LABORATÓRIOS

Apresentação referente à disciplina de Programação Orientada a Objetos para Web II, do Curso Superior de Tecnologia em Sistemas para Internet, da Universidade Federal de Santa Maria, cursada em regime de autodidatismo.

Santa Maria, RS

2026

1 CONTEXTO E ORIGINALIDADE

O SisBolsa é uma API REST desenvolvida para a disciplina de Programação Orientada a Objetos para Web II, cobrindo toda a gestão de bolsistas de pesquisa de uma instituição: cadastro, vínculos, frequência, relatórios e auditoria, expostos por uma API em Spring Boot 4 / Java 21. O projeto conta com 101 testes automatizados, documentação OpenAPI/Swagger e implantação via Docker — é a mesma API construída na disciplina POOW1, agora com foco total em backend

Repositório do sistema:

<https://github.com/matheuskotayko/sisBolsa>

2 MOTIVAÇÃO

O projeto foi criado pensando em minha bolsa anterior, quando trabalhava no setor de realidade virtual e inteligencia artificial no CTISM, a professora sempre exigia relatórios com pequenas entregas de tasks

- a) **Controle disperso em planilhas:** cada professor e laboratório mantém seus próprios registros, sem consolidação institucional
- b) **Sem rastreabilidade das horas:** apontamentos em papel ou e-mail não geram comprovante nem trilha auditável
- c) **Ocupação de laboratórios não visível:** não havia como saber, em tempo real, quantas vagas restam em cada laboratório
- d) **Ausência de trilha de auditoria:** alterações em cadastros e vínculos não ficavam registradas, dificultando a prestação de contas

3 OBJETIVOS

3.1 OBJETIVO GERAL

Desenvolver uma API REST completa para a gestão de bolsistas de pesquisa, cobrindo o cadastro de professores, laboratórios e projetos, o vínculo de bolsistas, o controle de frequência com emissão de comprovante em PDF, a geração de relatórios e exportações em CSV, e uma trilha de auditoria completa, com controle de acesso por perfil.

3.2 OBJETIVOS ESPECÍFICOS

- a) Adotar arquitetura stateless (sem sessão HTTP no servidor), com autenticação via token JWT reconstruído a cada requisição a partir do próprio token
- b) Implementar autenticação segura com JWT em cookie httpOnly, BCrypt e rate limiting anti-força-bruta
- c) Modelar e persistir o domínio com Spring Data JPA/Hibernate sobre PostgreSQL, versionado com Flyway
- d) Validar dados de entrada com Bean Validation, padronizando as respostas de erro por campo
- e) Seguir a semântica correta de métodos e códigos HTTP (PATCH para atualização parcial, PUT para substituição total, 201 Created com header Location, exceções de domínio mapeadas para status HTTP específicos)

4 RESUMO DO PROJETO

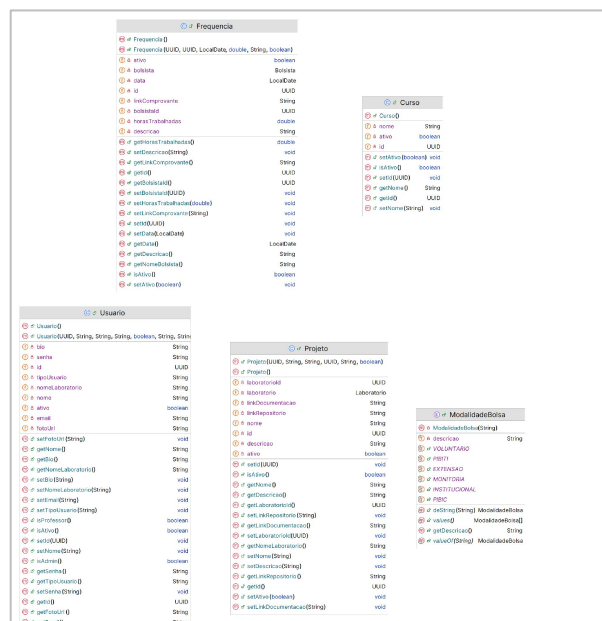
O SisBolsa é uma API REST em Java/Spring Boot para a gestão de bolsistas de pesquisa: cadastro de professores, laboratórios, bolsistas e projetos, controle de frequência com emissão de comprovante em PDF, relatórios agregados e uma trilha de auditoria completa, com autorização por papel de acesso (ADMIN, PROFESSOR, BOLSISTA).

- a) **Base da API:** /api/v1, com 8 controllers e 48 endpoints
- b) **Recursos principais:** usuários, laboratórios, projetos, frequências, relatórios, auditoria e cursos
- c) **Formato:** JSON, com paginação e filtros nos endpoints de listagem
- d) **Consumidores:** qualquer cliente HTTP — a API não pressupõe um frontend específico
- e) **Documentação interativa:** OpenAPI 3 / Swagger UI, gerada a partir do próprio código

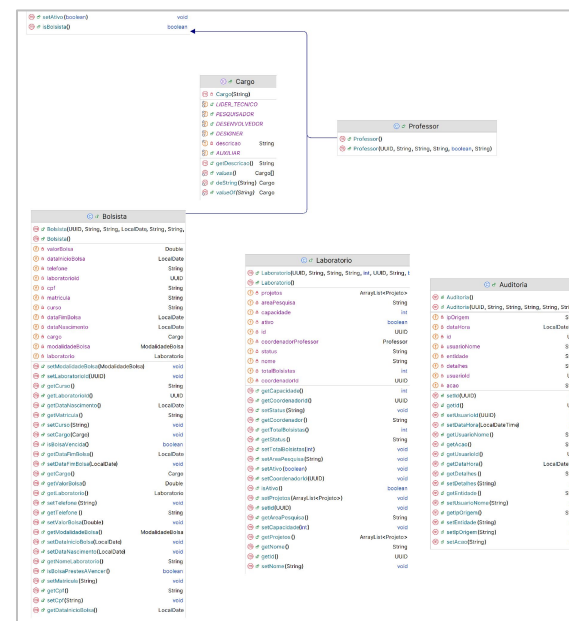
5.1 DIAGRAMA DE CLASSES

Hierarquia de entidades do domínio: **Usuario** é uma classe abstrata (**@MappedSuperclass**) especializada em **Bolsista** e **Professor**, associadas a Laboratório, Projeto, Frequência e Auditoria.

Figura 1 – Diagrama de classes do domínio



Parte 1 de 2 — Frecuencia, Curso, Usuario, Projeto, ModalidadeBolsa



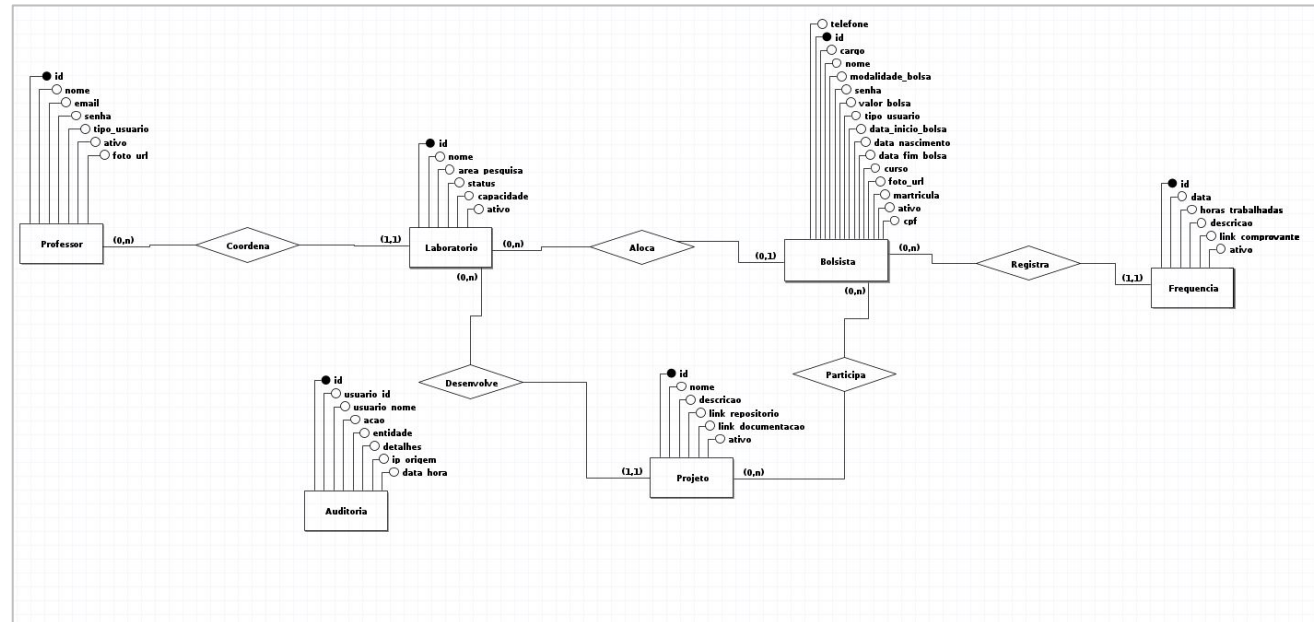
Parte 2 de 2 — *Cargo, Professor, Bolsista, Laboratório, Auditoria*

Fonte: Elaborado pelo autor (2026).

5.2 MODELAGEM DE DADOS — MODELO CONCEITUAL

O modelo conceitual organiza o domínio em seis entidades principais: Professor, Laboratório, Bolsista, Projeto, Frequência e Auditoria. Um professor coordena laboratórios e orienta projetos; bolsistas são alocados a laboratórios e vinculados a projetos (relação N:N); cada apontamento de frequência pertence a um bolsista, e toda ação relevante do sistema gera um registro na auditoria.

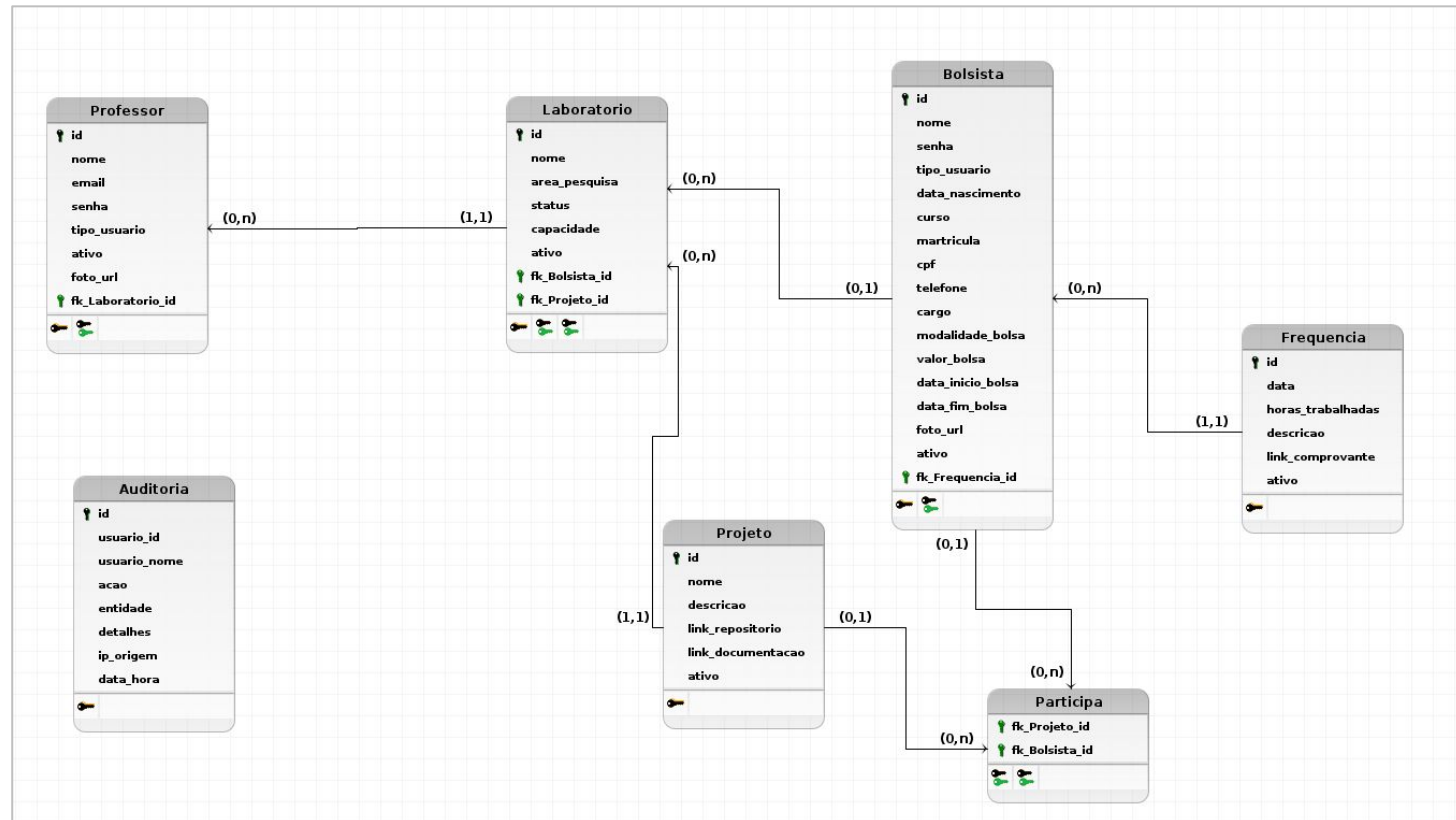
Figura 2 – Modelo conceitual (MER)



Fonte: Elaborado pelo autor (2026).

5.3 MODELAGEM DE DADOS — MODELO FÍSICO

Figura 3 – Modelo lógico (DER)



Fonte: Elaborado pelo autor (2026).

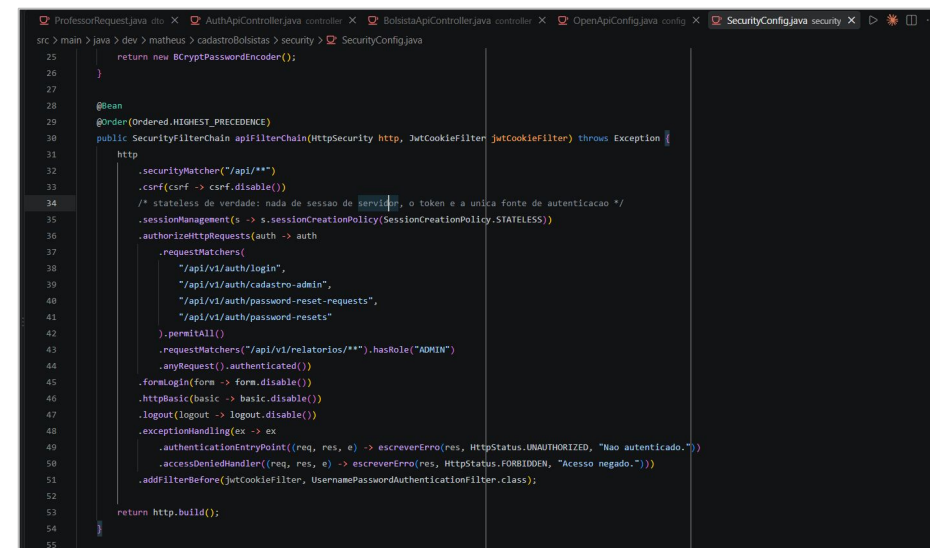
6 ARQUITETURA

- a) API REST em Spring Boot, seguindo os princípios REST: recursos via URIs, verbos HTTP para as operações CRUD, comunicação stateless e representações em JSON
- b) Camadas: Controller (endpoints) → Service (regras de negócio) → Repository (Spring Data JPA) → PostgreSQL, com DTOs isolando o contrato da API
- c) JWT em cookie httpOnly, BCrypt nas senhas e rate limiting anti-força-bruta; autorização por papel (ADMIN / PROFESSOR / BOLSISTA) via Spring Security
- d) Spring Data JPA/Hibernate sobre PostgreSQL 15, schema versionado com Flyway; API documentada via springdoc/OpenAPI 3 (Swagger UI)
- e) API consumida por qualquer cliente HTTP (não pressupõe frontend específico): dados trafegam em JSON, autenticação via token, sem estado de sessão guardado no servidor

BACKEND

Spring Boot 4.0.6 — Java 21 — Spring Data JPA/Hibernate — Spring Security — springdoc/Swagger — Docker Compose

Figura 4 – SecurityConfig.java



```
25 return new BCryptPasswordEncoder();
26 }
27
28 @Bean
29 @Order(Ordered.HIGHEST_PRECEDENCE)
30 public SecurityFilterChain apiFilterChain(HttpSecurity http, JwtCookieFilter jwtCookieFilter) throws Exception {
31     http
32         .securityMatchers("/api/**")
33         .csrf(csrf -> csrf.disable())
34         /* stateless de verdade: nada de sessão de servidor, o token é a única fonte de autenticação */
35         .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
36         .authorizeHttpRequests(auth -> auth
37             .requestMatchers(
38                 "/api/v1/auth/login",
39                 "/api/v1/auth/cadastro-admin",
40                 "/api/v1/auth/password-reset-requests",
41                 "/api/v1/auth/password-resets"
42             ).permitAll()
43             .requestMatchers("/api/v1/relatorios/**").hasRole("ADMIN")
44             .anyRequest().authenticated()
45         )
46         .formLogin(form -> form.disable())
47         .httpBasic(basic -> basic.disable())
48         .logout(logout -> logout.disable())
49         .exceptionHandling(ex -> ex
50             .authenticationEntryPoint((req, res, e) -> escreverErro(res, HttpStatus.UNAUTHORIZED, "Não autenticado."))
51             .accessDeniedHandler((req, res, e) -> escreverErro(res, HttpStatus.FORBIDDEN, "Acesso negado."))
52         )
53         .addFilterBefore(jwtCookieFilter, UsernamePasswordAuthenticationFilter.class);
54
55     return http.build();
56 }
```

SecurityConfig.java — sessão stateless, sem CSRF, filtro JWT

Fonte: Elaborado pelo autor (2026).

7 CAMADA DE CONTROLLER — BOAS PRÁTICAS REST

a) **Um controller por recurso**: mapeado em `/api/v1/<recurso>`, com verbos HTTP correspondentes às operações CRUD (GET, POST, PUT, PATCH, DELETE)

b) **PATCH para atualização parcial** (ex.: perfil do usuário) e **PUT para substituição total** (ex.: frequência, laboratório, projeto)

c) **201 Created com header Location** na criação (POST), montado via `UriComponentsBuilder` a partir do ID gerado

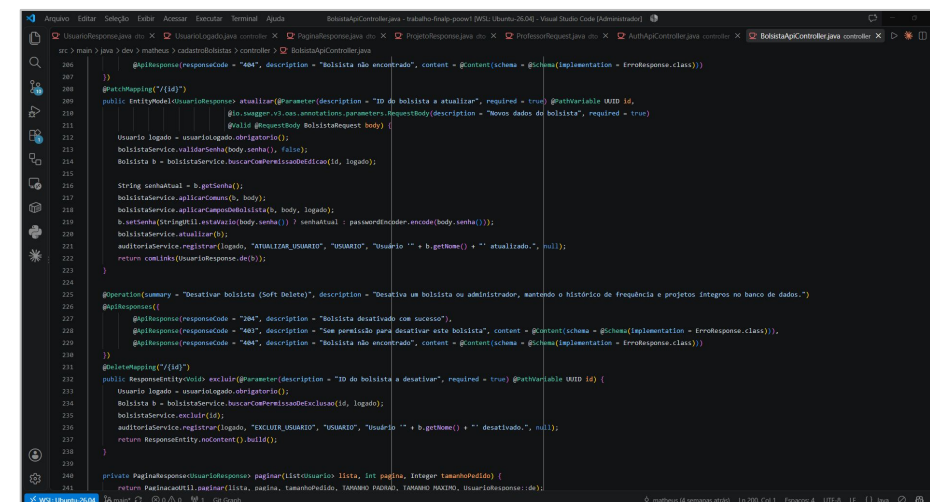
d) **Exclusão como soft delete**: DELETE retorna 204 No Content e desativa o registro, preservando o histórico

e) **Bean Validation nos DTOs**: `@Valid` + `@NotBlank`/`@Email`, com respostas de erro padronizadas por campo (400)

f) **Exceções de domínio mapeadas para status HTTP** via `@RestControllerAdvice` — 404, 403, 409, 401, 429, conforme o caso

g) **Respostas HATEOAS**: recursos individuais retornam `EntityModel` com links de navegação (self, recursos relacionados)

Figura 5 – `BolsistaApiController.java`



BolsistaApiController.java — métodos PATCH (atualizar) e DELETE (soft delete)

Fonte: Elaborado pelo autor (2026).

8 PERSISTÊNCIA — JPA E SPRING DATA

- a) **Entidades JPA:** `@Entity/@Table` por tabela, chave primária UUID gerada pelo próprio banco (`GenerationType.UUID`)
- b) **Herança sem herança de tabela:** `Usuario` é `@MappedSuperclass`; `Bolsista` e `Professor` são entidades concretas com tabelas próprias
- c) **Repositórios Spring Data JPA:** um `JpaRepository` por entidade, com queries derivadas por nome de método
- d) **Relacionamentos @ManyToOne somente leitura** (`insertable/updatable = false`): enriquecem DTOs de resposta sem abrir brecha de integridade via JPA
- e) **Schema versionado com Flyway:** cada alteração estrutural é uma migration numerada, aplicada automaticamente na subida da aplicação
- f) **Banco de dados:** PostgreSQL 15, com Hibernate como provider JPA

Figura 6 – Capturas de código: entidades e repositório JPA

Fonte: Elaborado pelo autor (2026).

*Usuario.java — superclasse
@MappedSuperclass*

Bolsista.java — @Entity, @ManyToOne somente leitura

BolsistaRepository.java — clique para ampliar

BolsistaRepository.java

```
package dev.matheus.cadastroBolsistas.repository;

import ...

@Repository 9 usages  matheus
public interface BolsistaRepository extends JpaRepository<Bolsista, UUID> {

    Optional<Bolsista> findByEmailAndAtivoTrue(String email); 6 usages  matheus

    List<Bolsista> findByAtivoTrueOrderByNome(); 1 usage  matheus

    List<Bolsista> findByNomeContainingIgnoreCaseAndAtivoTrueOrderByNome(String nome); 1 usage  matheus

    List<Bolsista> findByCursoContainingIgnoreCaseAndAtivoTrueOrderByNome(String curso); 1 usage  matheus

    @Query("SELECT b FROM Bolsista b WHERE b.laboratorioId = :labId AND b.ativo = true ORDER BY b.nome") 3 usages  matheus
    List<Bolsista> buscarPorLaboratorio(@Param("labId") UUID labId);

    @Query(value = "SELECT b.* FROM bolsista b " 1 usage  matheus
        + "INNER JOIN bolsista_projeto bp ON b.id = bp.bolsista_id "
        + "WHERE bp.projeto_id = :projetoId AND b.ativo = true ORDER BY b.nome",
        nativeQuery = true)
    List<Bolsista> buscarPorProjeto(@Param("projetoId") UUID projetoId);

    int countByTipoUsuarioAndAtivoTrue(String tipoUsuario); 7 usages  matheus

    @Modifying  matheus
    @Query("UPDATE Bolsista b SET b.ativo = false WHERE b.id = :id")
    int desativar(@Param("id") UUID id);
}
```

Clique em qualquer lugar para voltar

9 ENDPOINTS DA API — DOCUMENTAÇÃO SWAGGER

Quadro 1 – Endpoints da API

Recurso	Base da URL	Principais operações
Bolsistas	/api/v1/bolsistas	Listagem paginada e busca (nome/curso), cadastro, atualização parcial (PATCH), desativação, exportação CSV, cargos/modalidades — cobre também administradores (tipoUsuario=ADMIN)
Professores	/api/v1/professores	CRUD completo (listagem, busca, cadastro, atualização, desativação) — restrito a Administradores
Laboratórios	/api/v1/laboratorios	CRUD completo; bolsistas e projetos vinculados
Projetos	/api/v1/projetos	CRUD completo; vincular/desvincular bolsista da equipe
Frequências	/api/v1/frequencias	Registro, atualização, resumo mensal, exportação CSV, comprovante em PDF
Relatórios	/api/v1/relatorios	Indicadores agregados: ocupação, horas, distribuição por cargo
Auditoria	/api/v1/auditoria	Consulta e exportação de logs de segurança e operações
Autenticação	/api/v1/auth	Login, logout, perfil próprio, cadastro de admin, recuperação de senha
Cursos	/api/v1/cursos	Listagem e cadastro de cursos disponíveis

a) **Coleção completa (48 endpoints)** documentada interativamente em /swagger-ui.html (OpenAPI 3) e disponível como coleção Postman no repositório (SisBolsa-API + casos negativos)

10 SEGURANÇA E CONTROLE DE ACESSO

Quadro 2 – Perfis de acesso e permissões

Papel	Permissões
ADMIN	Acesso irrestrito: todos os laboratórios, bolsistas e projetos, relatórios globais, auditoria completa e configurações do sistema
PROFESSOR	Gerencia os laboratórios que coordena, os projetos associados e os bolsistas vinculados a eles
BOLSISTA	Apontamento e edição da própria frequência, emissão de comprovantes em PDF, visualização de equipe e projetos

a) **Sessão:** token JWT guardado em cookie httpOnly, inacessível a JavaScript no navegador

b) **Rate limiting:** 5 tentativas de login malsucedidas bloqueiam a conta por 5 minutos

c) **Recuperação de senha:** código numérico de 6 dígitos, válido por 15 minutos

d) **Medidor de força de senha:** feedback visual em tempo real no cadastro e na troca de senha

11 TESTES AUTOMATIZADOS

A suíte conta com 101 testes automatizados — unitários e de contexto com MockMvc — que não dependem de banco de dados externo, cobrindo regras de negócio, validações de domínio e a matriz de autorização por papel de acesso.

CRITÉRIOS DE AVALIAÇÃO

- a) **Controle de acesso granular:** três perfis com permissões efetivamente distintas, não apenas cosméticas
- b) **Segurança concreta:** JWT em cookie httpOnly, BCrypt e rate limiting anti-força-bruta
- c) **Auditoria completa:** toda ação relevante do sistema fica registrada, com filtros e exportação
- d) **Geração nativa de PDF:** comprovante de frequência com layout institucional, sem dependências externas
- e) **Documentação viva:** API documentada via OpenAPI 3/Swagger UI, gerada a partir do próprio

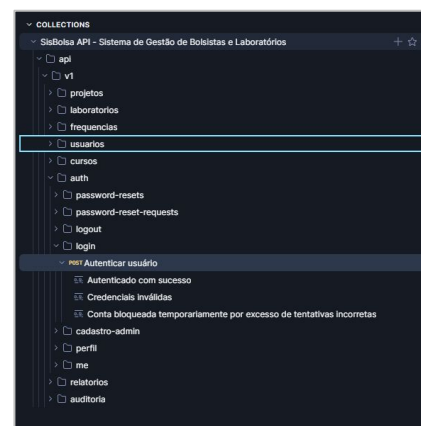
12 DEMONSTRAÇÃO

A demonstração é feita diretamente sobre a API: o Swagger UI documenta e permite testar os 48 endpoints a partir do próprio código, e as coleções Postman reproduzem os principais fluxos — login, CRUD de bolsistas e projetos, frequência, relatórios e auditoria.

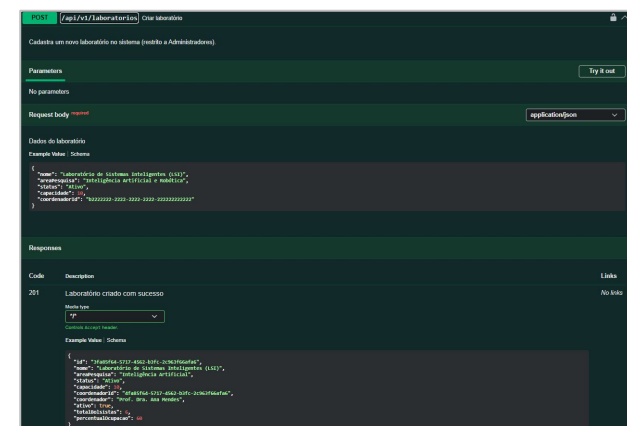
Figura 7 – Swagger UI e coleção Postman



Swagger UI — `/swagger-ui.html` — documentação interativa (OpenAPI 3)



Coleção Postman — organizada por recurso (`/api/v1/...`)



Try it out — chamada real a um endpoint a partir do Swagger

Fonte: Elaborado pelo autor (2026).

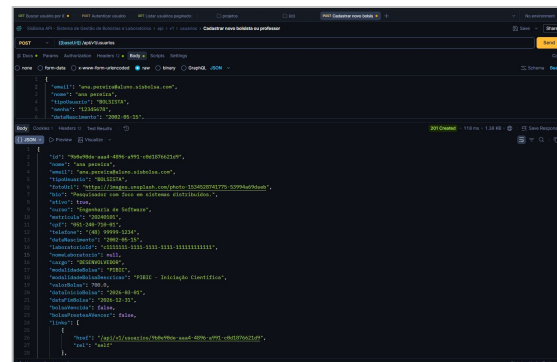
Coleção completa: github.com/matheuskotayko/sisBolsa/postman

12.1 DEMONSTRAÇÃO — SUCESSO E CASOS NEGATIVOS

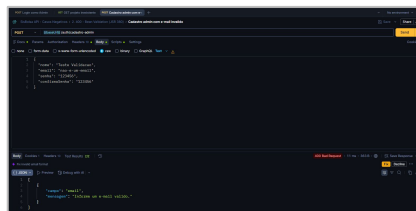
Cada resposta abaixo corresponde a uma exceção de domínio mapeada no ApiExceptionHandler (slide 7), testada na coleção Postman de casos negativos.

Figura 8 – Respostas de sucesso e casos negativos

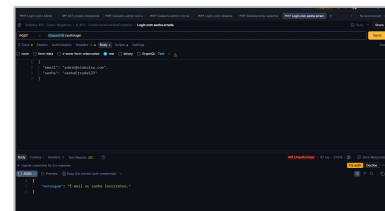
Fonte: Elaborado pelo autor (2026).



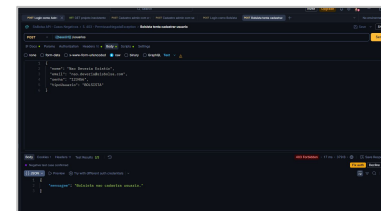
201 Created — *cadastro de bolsista, resposta com HATEOAS*



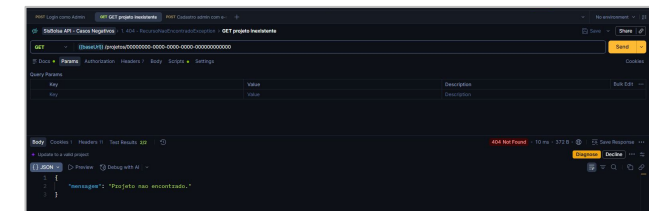
400 — *Bean Validation (e-mail inválido)*



401 — *credenciais inválidas no login*



403 — *bolsista sem permissão para cadastrar usuário*



404 — *projeto inexistente*

13 CONSIDERAÇÕES FINAIS

O SisBolsa entrega uma API REST completa para gestão de bolsistas de pesquisa, com autenticação e controle de acesso concretos, persistência versionada, comprovantes em PDF gerados nativamente e uma trilha de auditoria que cobre todas as ações do sistema — características pensadas para uso real, e não apenas para atender a um requisito acadêmico isolado.

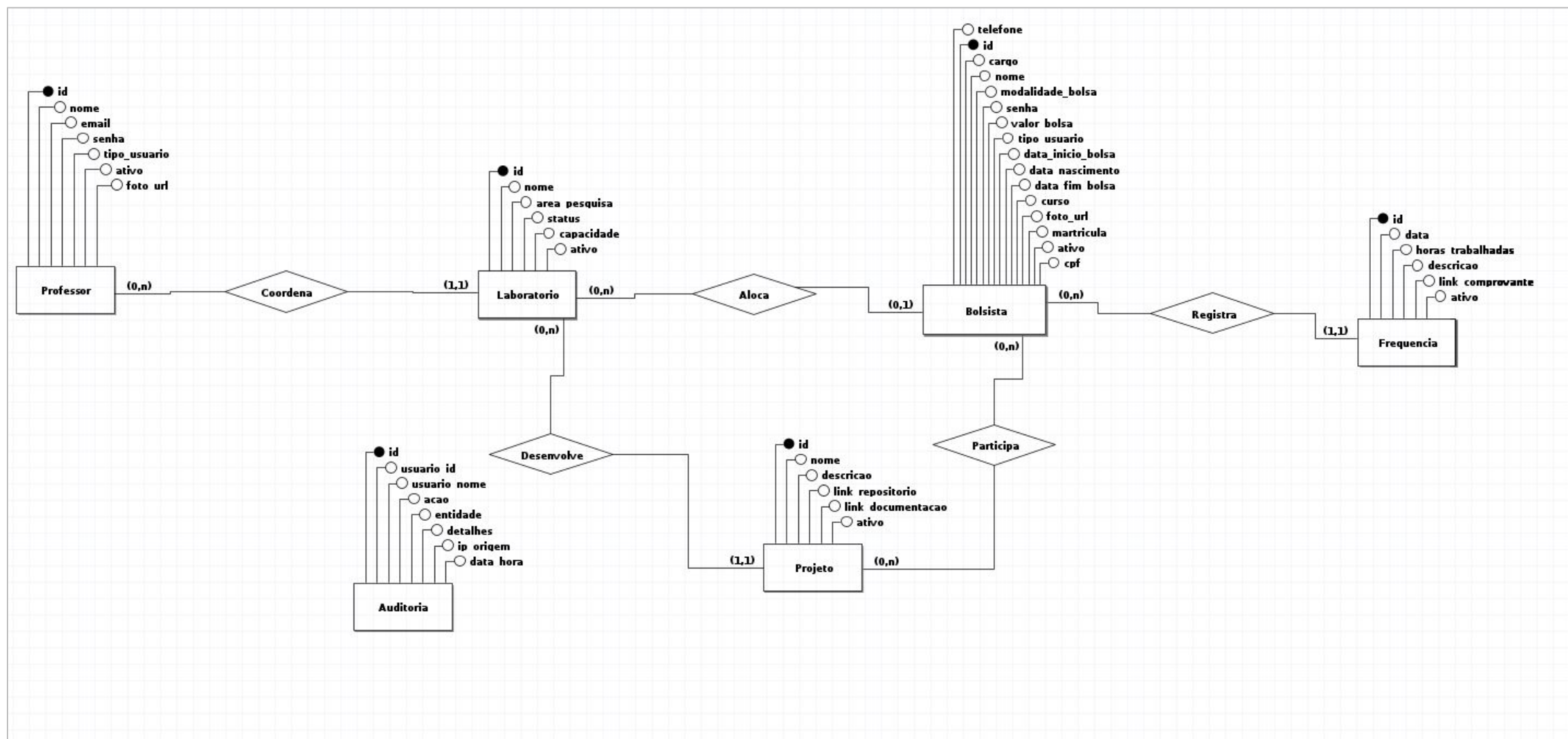
O que pode ser implementado futuramente

- a) Notificações por e-mail para prazos de entregáveis dos projetos
- b) Aplicativo mobile para apontamento de frequência em campo
- c) Assinatura digital dos comprovantes de frequência em PDF

Repositório do sistema:

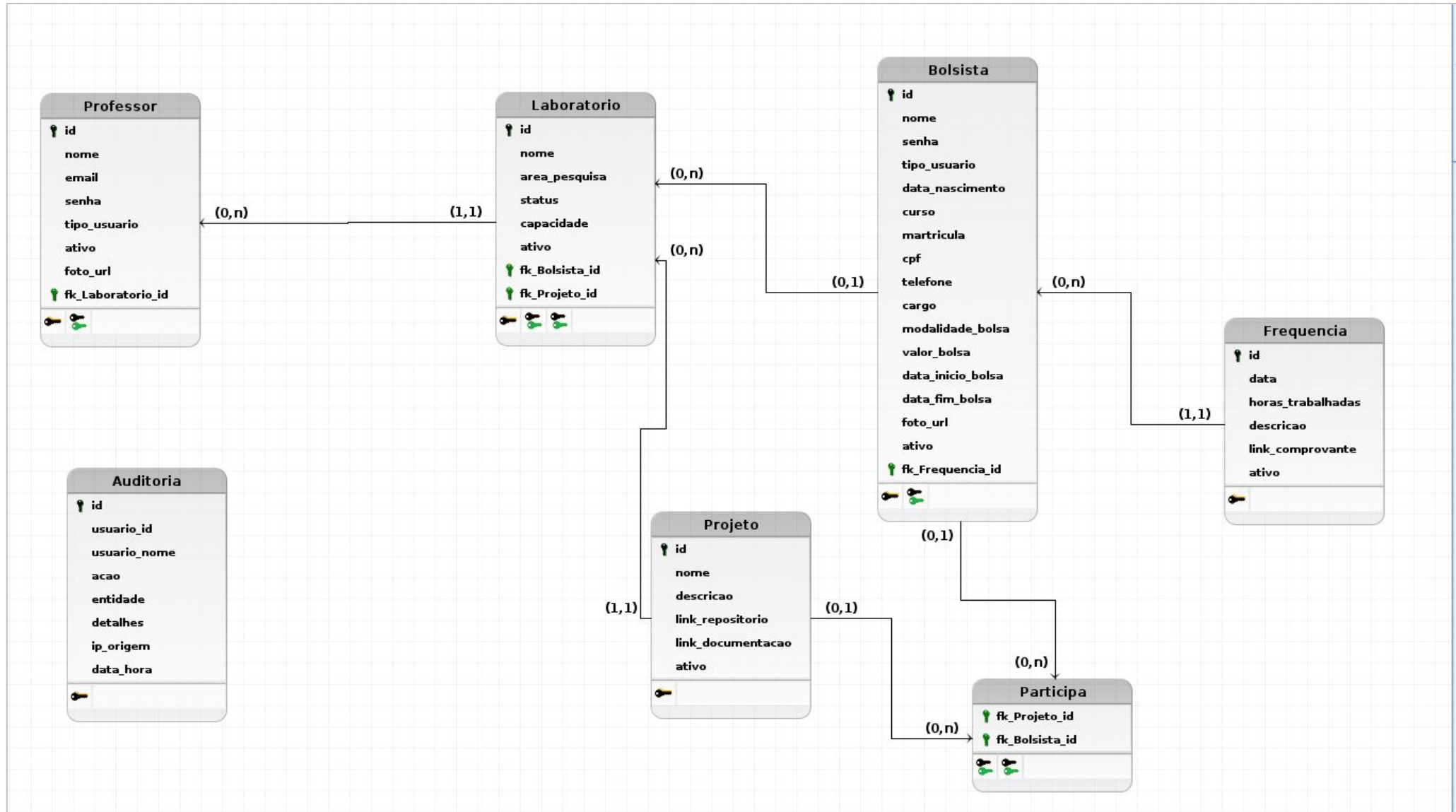
<https://github.com/matheuskotayko/sisBolsa>

Diagrama Entidade-Relacionamento (DER)



Clique em qualquer lugar para voltar

Modelo Lógico do Banco de Dados



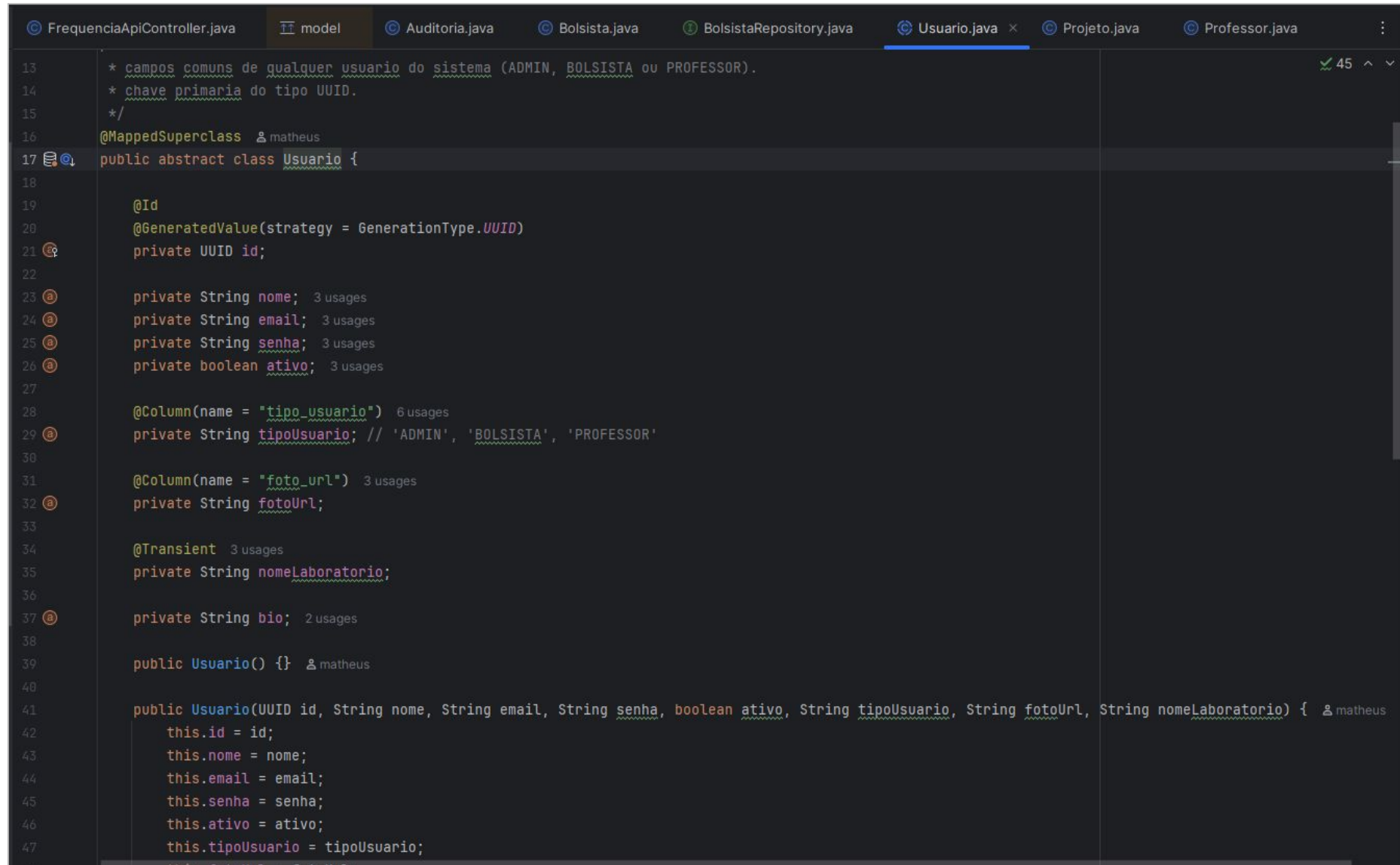
Clique em qualquer lugar para voltar

SecurityConfig.java

```
ProfessorRequest.java dto X AuthApiController.java controller X BolsistaApiController.java controller X OpenApiConfig.java config X SecurityConfig.java security X ▶ ⚙️ □ ...
src > main > java > dev > matheus > cadastroBolsistas > security > SecurityConfig.java
25     return new BCryptPasswordEncoder();
26 }
27
28 @Bean
29 @Order(Ordered.HIGHEST_PRECEDENCE)
30 public SecurityFilterChain apiFilterChain(HttpSecurity http, JwtCookieFilter jwtCookieFilter) throws Exception {
31     http
32         .securityMatcher("/api/**")
33         .csrf(csrf -> csrf.disable())
34         /* stateless de verdade: nada de sessao de servidor, o token e a unica fonte de autenticacao */
35         .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
36         .authorizeHttpRequests(auth -> auth
37             .requestMatchers(
38                 "/api/v1/auth/login",
39                 "/api/v1/auth/cadastro-admin",
40                 "/api/v1/auth/password-reset-requests",
41                 "/api/v1/auth/password-resets"
42             ).permitAll()
43             .requestMatchers("/api/v1/relatorios/**").hasRole("ADMIN")
44             .anyRequest().authenticated()
45         .formLogin(form -> form.disable())
46         .httpBasic(basic -> basic.disable())
47         .logout(logout -> logout.disable())
48         .exceptionHandling(ex -> ex
49             .authenticationEntryPoint((req, res, e) -> escreverErro(res, HttpStatus.UNAUTHORIZED, "Nao autenticado."))
50             .accessDeniedHandler((req, res, e) -> escreverErro(res, HttpStatus.FORBIDDEN, "Acesso negado.)))
51         .addFilterBefore(jwtCookieFilter, UsernamePasswordAuthenticationFilter.class);
52
53     return http.build();
54 }
55
```

Clique em qualquer lugar para voltar

Usuario.java



```
13  * campos comuns de qualquer usuario do sistema (ADMIN, BOLSISTA ou PROFESSOR).
14  * chave primaria do tipo UUID.
15  */
16  @MappedSuperclass
17  public abstract class Usuario {
18
19      @Id
20      @GeneratedValue(strategy = GenerationType.UUID)
21      private UUID id;
22
23      private String nome;
24      private String email;
25      private String senha;
26      private boolean ativo;
27
28      @Column(name = "tipo_usuario")
29      private String tipoUsuario;
30
31      @Column(name = "foto_url")
32      private String fotoUrl;
33
34      @Transient
35      private String nomeLaboratorio;
36
37      private String bio;
38
39      public Usuario() {}
40
41      public Usuario(UUID id, String nome, String email, String senha, boolean ativo, String tipoUsuario, String fotoUrl, String nomeLaboratorio) {
42          this.id = id;
43          this.nome = nome;
44          this.email = email;
45          this.senha = senha;
46          this.ativo = ativo;
47          this.tipoUsuario = tipoUsuario;
```

Clique em qualquer lugar para voltar

Bolsista.java

```
18 @Entity & matheus
19 @Table(name = "bolsista")
20 public class Bolsista extends Usuario {
21
22     @Column(name = "data_nascimento") 3 usages
23     private LocalDate dataNascimento;
24
25     @Column(name = "curso") 3 usages
26     private String curso;
27     @Column(name = "matricula") 3 usages
28     private String matricula;
29     @Column(name = "cpf") 3 usages
30     private String cpf;
31     @Column(name = "telefone") 3 usages
32     private String telefone;
33
34     @Column(name = "laboratorio_id") 3 usages
35     private UUID laboratorioId;
36
37     @ManyToMany 3 usages
38     @JoinColumn(name = "laboratorio_id", insertable = false, updatable = false)
39     private Laboratorio laboratorio;
40
41     @Enumerated(EnumType.STRING) 3 usages
42     private Cargo cargo;
43
44     @Enumerated(EnumType.STRING) 2 usages
45     @Column(name = "modalidade_bolsa")
46     private ModalidadeBolsa modalidadeBolsa;
47
48     @Column(name = "valor_bolsa") 2 usages
49     private Double valorBolsa;
50
51     @Column(name = "data_inicio_bolsa") 2 usages
52     private LocalDate dataInicioBolsa;
53
54     @Column(name = "data_fim_bolsa") 6 usages
55     private LocalDate dataFimBolsa;
56
57     public Bolsista() { & matheus
58         super();
59     }
60 }
```

Clique em qualquer lugar para voltar

Swagger UI — Documentação

SisBolsa API - Sistema de Gestão de Bolsistas e Laboratórios 1.0.0 OAS 3.1

/v3/api-docs

Visão Geral

API RESTful para gerenciamento completo de bolsistas, professores, laboratórios de pesquisa, projetos acadêmicos, controle de frequência/horas e trilha de auditoria.

Autenticação e Segurança

- Autenticação Baseada em JWT: Para autenticar, utilize `POST /api/auth/login`. A API grava automaticamente um cookie `httpOnly ( token )` com proteção `SameSite=Strict`.
- Rate Limiting & Anti-Brute Force: Limite de 5 tentativas consecutivas com erro. Em caso de excesso, a conta é bloqueada temporariamente por 5 minutos (HTTP 429).
- Controle de Acesso Baseado em Perfis (RBAC):
 - `ADMIN` : Acesso total irrestrito a todos os recursos, auditoria, configurações e relatórios globais.
 - `PROFESSOR` : Gerenciamento dos laboratórios que coordena, seus projetos associados e bolsistas vinculados.
 - `BOLSISTA` : Apontamento de frequência própria, visualização do seu laboratório, projetos e comprovantes em PDF.

[Contact Suporte SisBolsa](#)

[MIT License](#)

Servers

`http://localhost:8080` - Generated server url

Authorize



Auditoria Trilha de auditoria para rastreamento de acessos, alterações de cadastros, logins e emissões de comprovantes.



GET

`/api/v1/auditoria` Listar logs de auditoria paginados



GET

`/api/v1/auditoria/exportar` Exportar logs de auditoria em CSV



Autenticação Endpoints para autenticação, controle de sessão via JWT HttpOnly, perfil e recuperação de senha.



Bolsistas & Usuários Gestão de bolsistas, professores e administradores, incluindo vigência, modalidades e cargos.



Cursos Lista de cursos disponíveis para vínculo de bolsistas.



Frequência & Horas Apontamento de horas trabalhadas, resumo mensal, exportação em CSV e emissão de comprovantes em PDF.

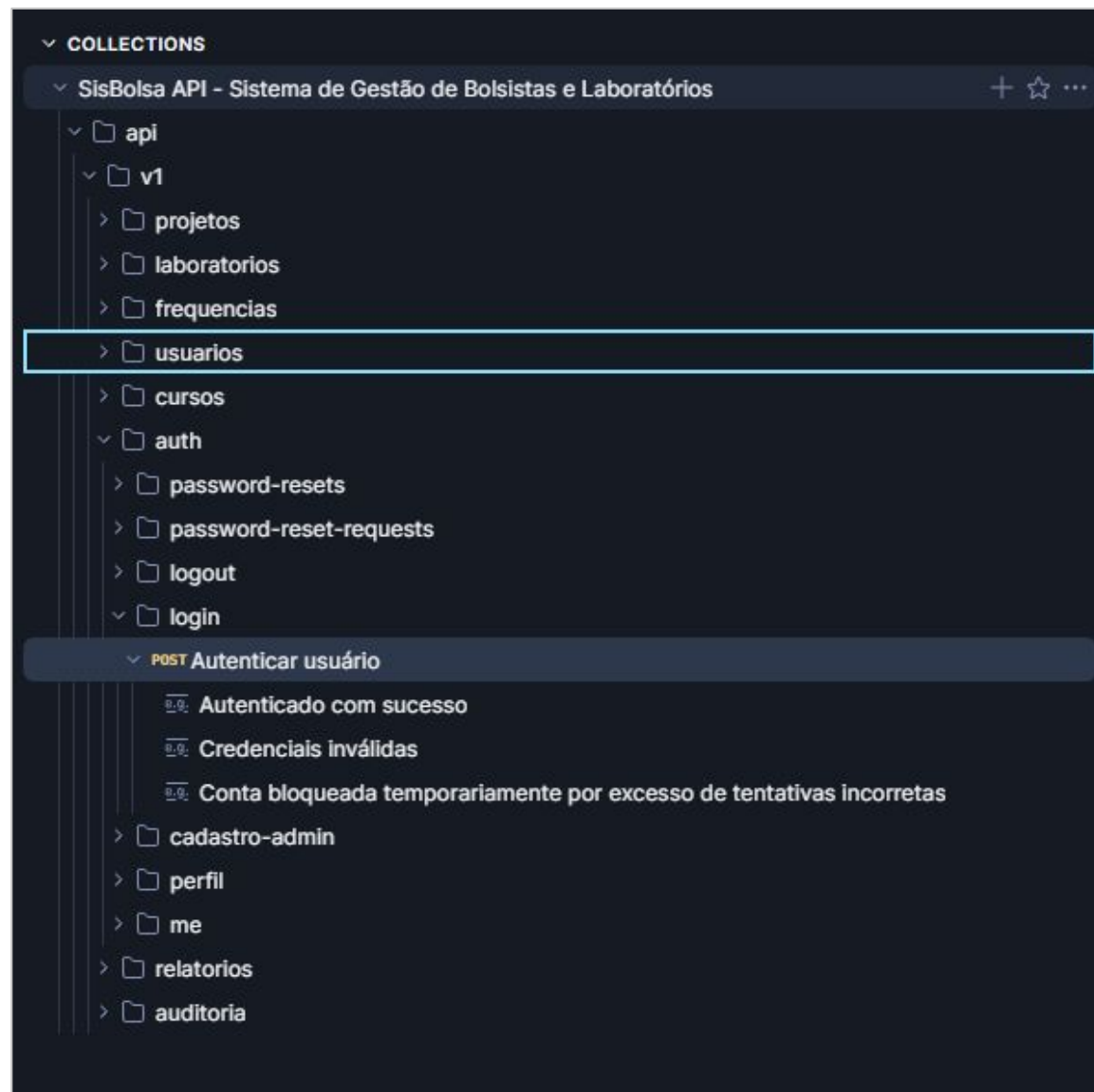


Laboratórios Gerenciamento de laboratórios de pesquisa, ensino, extensão, pesquisa e ensino de pesquisa.



Clique em qualquer lugar para voltar

Coleção Postman



Clique em qualquer lugar para voltar

Swagger — Try it out

POST

/api/v1/laboratorios

Criar laboratório

🔒 ⬆

Cadastra um novo laboratório no sistema (restrito a Administradores).

Parameters

Try it out

No parameters

Request body required

application/json ⌵

Dados do laboratório

Example Value | Schema

```
{
  "nome": "Laboratório de Sistemas Inteligentes (LSI)",
  "areaPesquisa": "Inteligência Artificial e Robótica",
  "status": "Ativo",
  "capacidade": 10,
  "coordenadorId": "b2222222-2222-2222-2222-222222222222"
}
```

Responses

Code	Description	Links
201	Laboratório criado com sucesso	No links

Media type

/ ⌵

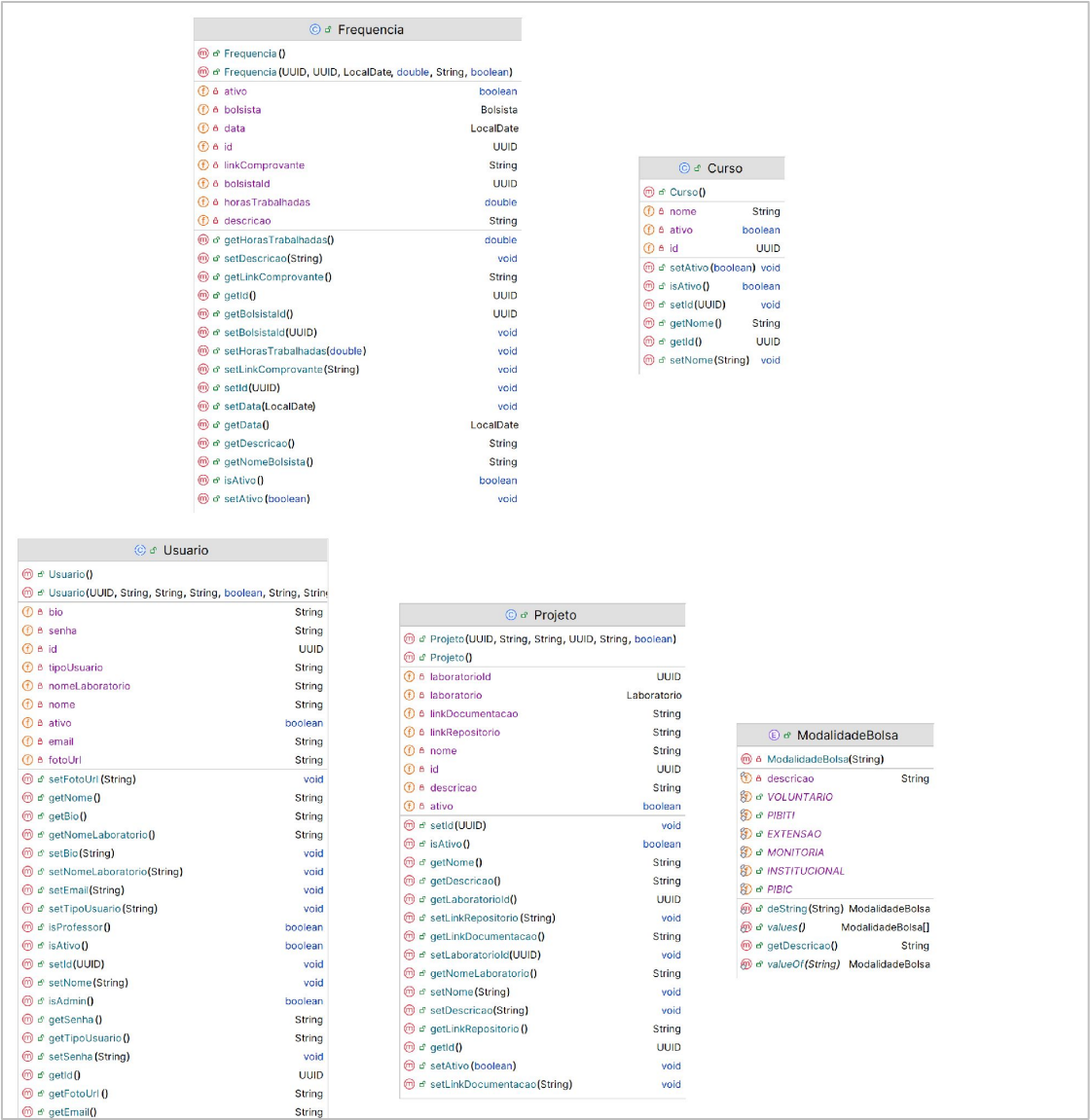
Controls Accept header.

Example Value | Schema

```
{
  "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "nome": "Laboratório de Sistemas Inteligentes (LSI)",
  "areaPesquisa": "Inteligência Artificial",
  "status": "Ativo",
  "capacidade": 10,
  "coordenadorId": "4fa85f64-5717-4562-b3fc-2c963f66afa6",
  "coordenador": "Prof. Dra. Ana Mendes",
  "ativo": true,
  "totalBolsistas": 6,
  "percentualOcupacao": 60
}
```

Clique em qualquer lugar para voltar

Diagrama de Classes — Parte 1 de 2



Clique em qualquer lugar para voltar

Diagrama de Classes — Parte 2 de 2



Clique em qualquer lugar para voltar

BolsistaApiController.java

```
Arquivo Editar Seleção Exibir Acessar Executar Terminal Ajuda BolsistaApiController.java - trabalho-finalp-poo1 [WSL: Ubuntu-26.04] - Visual Studio Code [Administrador]

UsuarioResponse.java dto X UsuarioLogado.java controller X PaginaResponse.java dto X ProjetoResponse.java dto X ProfessorRequest.java dto X AuthApiController.java controller X BolsistaApiController.java controller X

src > main > java > dev > matheus > cadastroBolsistas > controller > BolsistaApiController.java

206         @ApiResponse(responseCode = "404", description = "Bolsista não encontrado", content = @Content(schema = @Schema(implementation = ErrorResponse.class)))
207     })
208     @PatchMapping("/{id}")
209     public EntityModel<UsuarioResponse> atualizar(@Parameter(description = "ID do bolsista a atualizar", required = true) @PathVariable UUID id,
210         @io.swagger.v3.oas.annotations.parameters.RequestBody(description = "Novos dados do bolsista", required = true)
211         @Valid @RequestBody BolsistaRequest body) {
212         Usuario logado = usuarioLogado.obrigatorio();
213         bolsistaService.validarSenha(body.senha(), false);
214         Bolsista b = bolsistaService.buscarComPermissaoDeEdicao(id, logado);
215
216         String senhaAtual = b.getSenha();
217         bolsistaService.aplicarComuns(b, body);
218         bolsistaService.aplicarCamposDeBolsista(b, body, logado);
219         b.setSenha(StringUtil.estaVazio(body.senha()) ? senhaAtual : passwordEncoder.encode(body.senha()));
220         bolsistaService.atualizar(b);
221         auditoriaService.registrar(logado, "ATUALIZAR_USUARIO", "USUARIO", "Usuário '" + b.getNome() + "' atualizado.", null);
222         return comLinks(UsuarioResponse.de(b));
223     }
224
225     @Operation(summary = "Desativar bolsista (Soft Delete)", description = "Desativa um bolsista ou administrador, mantendo o histórico de frequência e projetos íntegros no banco de dados.")
226     @ApiResponses({
227         @ApiResponse(responseCode = "204", description = "Bolsista desativado com sucesso"),
228         @ApiResponse(responseCode = "403", description = "Sem permissão para desativar este bolsista", content = @Content(schema = @Schema(implementation = ErrorResponse.class))),
229         @ApiResponse(responseCode = "404", description = "Bolsista não encontrado", content = @Content(schema = @Schema(implementation = ErrorResponse.class)))
230     })
231     @DeleteMapping("/{id}")
232     public ResponseEntity<Void> excluir(@Parameter(description = "ID do bolsista a desativar", required = true) @PathVariable UUID id) {
233         Usuario logado = usuarioLogado.obrigatorio();
234         Bolsista b = bolsistaService.buscarComPermissaoDeExclusao(id, logado);
235         bolsistaService.excluir(id);
236         auditoriaService.registrar(logado, "EXCLUIR_USUARIO", "USUARIO", "Usuário '" + b.getNome() + "' desativado.", null);
237         return ResponseEntity.noContent().build();
238     }
239
240     private PaginaResponse<UsuarioResponse> paginar(List<Usuario> lista, int pagina, Integer tamanhoPedido) {
241         return PaginacaoUtil.paginar(lista, pagina, tamanhoPedido, TAMANHO PADRAO, TAMANHO MAXIMO, UsuarioResponse::de);
242     }

WSL: Ubuntu-26.04 main* 0 1 Git Graph matheus (4 semanas atrás) Ln 200, Col 1 Espaços: 4 UTF-8 LF {} Java
```

Clique em qualquer lugar para voltar

201 Created — Cadastro de Bolsista

The screenshot displays a REST client interface with the following details:

- Request:**
 - Method: **POST**
 - URL: `{{baseUri}}/api/v1/usuarios`
 - Body (raw):

```
{
  "email": "ana.pereira@aluno.sisbolsa.com",
  "nome": "ana pereira",
  "tipoUsuario": "BOLSISTA",
  "senha": "12345678",
  "dataNascimento": "2002-05-15".
}
```
- Response:**
 - Status: **201 Created** (118 ms, 1.38 KB)
 - Body (JSON):

```
{
  "id": "9b0e90de-aaa4-4896-a991-c0d1876621d9",
  "nome": "ana pereira",
  "email": "ana.pereira@aluno.sisbolsa.com",
  "tipoUsuario": "BOLSISTA",
  "fotoUrl": "https://images.unsplash.com/photo-1534528741775-53994a69daeb",
  "bio": "Pesquisador com foco em sistemas distribuídos.",
  "ativo": true,
  "curso": "Engenharia de Software",
  "matricula": "20240101",
  "cpf": "051-240-710-01",
  "telefone": "(48) 99999-1234",
  "dataNascimento": "2002-05-15",
  "laboratorioId": "c1111111-1111-1111-1111-111111111111",
  "nomeLaboratorio": null,
  "cargo": "DESENVOLVEDOR",
  "modalidadeBolsa": "PIBIC",
  "modalidadeBolsaDescricao": "PIBIC - Iniciação Científica",
  "valorBolsa": 700.0,
  "dataInicioBolsa": "2026-03-01",
  "dataFimBolsa": "2026-12-31",
  "bolsaVencida": false,
  "bolsaPrestesAVencer": false,
  "links": [
    {
      "href": "/api/v1/usuarios/9b0e90de-aaa4-4896-a991-c0d1876621d9",
      "rel": "self"
    }
  ]
}
```

At the bottom of the interface, it states: "No anomalies detected!"

Clique em qualquer lugar para voltar

400 — Bean Validation

The screenshot shows a REST client interface with the following details:

- Top Bar:** Contains tabs for "POST Login como Admin", "GET GET projeto inexistente", and "POST Cadastro admin com e-mail invalido". The environment is set to "No environment".
- Breadcrumb:** "SisBolsa API - Casos Negativos > 2. 400 - Bean Validation (JSR 380) > Cadastro admin com e-mail invalido".
- Request Section:**
 - Method:** POST
 - URL:** `{{baseUrl}}/auth/cadastro-admin`
 - Body:** A JSON object:

```
{  "nome": "Teste Validacao",  "email": "nao-e-um-email",  "senha": "123456",  "confirmaSenha": "123456"}
```
- Response Section:**
 - Status:** 400 Bad Request (11 ms, 363 B)
 - Message:** Fix invalid email format
 - Body:** A JSON array:

```
[  {    "campo": "email",    "mensagem": "Informe um e-mail valido."  }]
```

Clique em qualquer lugar para voltar

401 — Credenciais Inválidas

The screenshot shows a REST client interface with a list of requests at the top. The selected request is a POST to `{{baseUrl}}/auth/login`. The request body is a JSON object with email and password. The response is a 401 Unauthorized status with a message in Portuguese. The interface includes tabs for Docs, Params, Authorization, Headers, Body, Scripts, and Settings. The Body tab is active, showing the raw JSON request and the JSON response.

Request:

```
POST {{baseUrl}}/auth/login
```

Request Body (raw):

```
1 {
2   "email": "admin@sisbolsa.com",
3   "senha": "senhaErrada123"
4 }
```

Response:

```
1 {
2   "mensagem": "E-mail ou senha incorretos."
3 }
```

Status: 401 Unauthorized • 57 ms • 376 B

Buttons: Fix auth, Decline

Clique em qualquer lugar para voltar

403 — Acesso Negado

The screenshot shows a REST client interface with a dark theme. At the top, there's a search bar and navigation links like 'Invite' and 'Upgrade'. Below that, a breadcrumb trail reads: 'SisBolsa API - Casos Negativos > 5, 403 - PermissaoNegadaException > Bolsista tenta cadastrar usuario'. The main area shows a POST request to '{{baseUrl}} /usuarios' with a JSON body:

```
{
  "nome": "Nao Deveria Existir",
  "email": "nao.deveria@sisbolsa.com",
  "senha": "123456",
  "tipoUsuario": "BOLSISTA"
}
```

. The response is a 403 Forbidden status with a message:

```
{
  "mensagem": "Bolsista nao cadastra usuario."
}
```

. The interface includes tabs for 'Body', 'Cookies', 'Headers', and 'Test Results'. A status bar at the bottom indicates 'Negative test case confirmed' and provides buttons for 'Fix auth' and 'Decline'.

POST Login como Adm × GET GET projeto inexistente POST Cadastro admin com e- POST Cadastro admin com se POST Login como Bolsista POST Bolsista tenta cadastrar + No environment ▼ Save Share

SisBolsa API - Casos Negativos > 5, 403 - PermissaoNegadaException > Bolsista tenta cadastrar usuario

POST ▼ {{baseUrl}} /usuarios Send ▼

Docs ● Params Authorization Headers 10 ● Body ● Scripts ● Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL Text ▼ ⚠

```
1 {
2   "nome": "Nao Deveria Existir",
3   "email": "nao.deveria@sisbolsa.com",
4   "senha": "123456",
5   "tipoUsuario": "BOLSISTA"
6 }
```

Body Cookies 1 Headers 11 Test Results 2/2 ↺ 403 Forbidden • 17 ms • 379 B • 🌐 Save Response ⋮

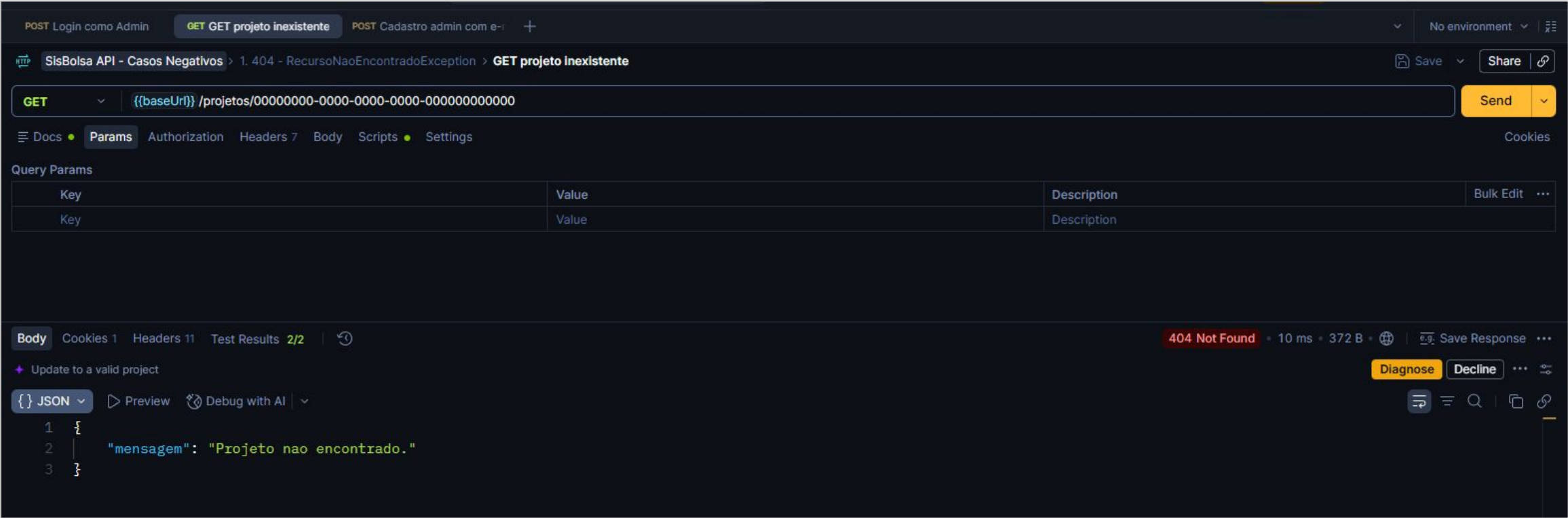
✖ Negative test case confirmed Fix auth Decline ⋮ 🔄

{ } JSON ▼ ▶ Preview ⚙ Try with different auth credentials ▼

```
1 {
2   "mensagem": "Bolsista nao cadastra usuario."
3 }
```

Clique em qualquer lugar para voltar

404 — Recurso Inexistente



Clique em qualquer lugar para voltar